

Khi test trượt là chưa đủ: đánh giá phân cụm quan niệm sai lầm trong bài lập trình C bằng nhãn từ bản sửa của chính sinh viên

Tóm tắt

Gom cụm các bài nộp sai theo những test case mà chúng trượt, rồi giải thích cụm bằng luật NẾU–THÌ, là một đề xuất phổ biến để cho giảng viên biết cả lớp đang hiểu sai ở đâu. Tuy vậy, câu hỏi liệu các cụm đó có gom chương trình theo *cơ chế* lỗi hay không hiếm khi được kiểm chứng, vì thiếu nhãn. Chúng tôi tự xây dựng nhãn mà không cần gán nhãn thủ công. Chạy lại 8.607 bài C90 của một bộ dữ liệu công khai trong môi trường cách ly (khớp 98,42% với hệ thống chấm lịch sử), chúng tôi thu gọn khác biệt giữa mỗi bài sai và lần nộp đạt kế tiếp của chính sinh viên đó thành một bản sửa tối thiểu đã kiểm chứng bằng delta debugging, rồi phân loại vào mười hai cơ chế mức chương trình; một benchmark thứ hai tiêm đúng một cơ chế vào các bài đã đạt. Theo protocol đăng ký trước, chữ ký test giống hệt nhau trộn lẫn nhiều cơ chế: không hàm nào của biểu diễn kết quả test có thể gán đúng quá 73,9% trong 339 bài thật và 50,7% trong 3.320 bài tiêm lỗi, và 81,6% các cặp bài tiêm lỗi có cùng chữ ký lại khác cơ chế. Các mô tả không cần nhãn về *cách* output lệch khỏi đáp án nâng trần này vượt mức một phân hoạch ngẫu nhiên cùng độ mịn, và trên dữ liệu kiểm soát nâng chỉ số Rand hiệu chỉnh của cụm K-means từ 0,09 lên 0,33; trên bản sửa thật, mức tăng (0,29 lên 0,35) không đáng tin cậy về thống kê và các mô tả này không vượt được quan hệ output đơn giản. Một mô hình embedding mã đóng băng gom mutant theo chương trình gốc chứ không theo cơ chế. Luật ILA-2 trên các thuộc tính độ lệch không phụ thuộc bài chuyển được sang bài tập mới của benchmark kiểm soát (macro-F1 0,35). Bằng chứng từ output, chứ không phải mô hình lớn hơn, là con đường rẻ nhất tới phản hồi ở mức cơ chế.

Từ khóa: giáo dục lập trình, quan niệm sai lầm, phân cụm, sửa lỗi chương trình, quy nạp luật, khai phá dữ liệu giáo dục

1. Giới thiệu

Giảng viên các lớp lập trình nhập môn quy mô lớn không thể đọc hết mọi bài nộp sai, nhưng vẫn muốn biết cả lớp đang sai ở đâu để giảng lại.

Một câu trả lời phổ biến là gom cụm các bài sai rồi tóm tắt từng cụm cho giảng viên. Trong cách phát biểu thường gặp nhất, mỗi bài nộp là một đối tượng được mô tả bằng các cặp thuộc tính–giá trị ghi lại test nào bị trượt (biểu diễn đối tượng–thuộc tính–giá trị, OAV); các đối tượng được gom bằng K-means hoặc một độ đo Hamming; sau đó quy nạp các luật sản xuất dạng *NẾU triệu chứng THÌ quan niệm sai*. Cách làm này hấp dẫn: kết quả test có sẵn ở mọi hệ thống chấm tự động, biểu diễn dễ giải thích, và luật đọc lên giống lời khuyên giảng dạy.

Câu hỏi liệu các cụm đó có gom bài theo *cơ chế* lỗi hay không hiếm khi được kiểm chứng, vì một lý do đơn giản: các bộ dữ liệu công khai không có nhãn cơ chế, và thuê chuyên gia gán nhãn hàng nghìn chương trình vừa tốn kém vừa khó tái lập. Khi thiếu nhãn, chất lượng cụm thường được đánh giá bằng chỉ số nội tại như silhouette, hoặc bằng độ khớp của luật thay thế với cụm. Cả hai đều không cho biết hai chương trình trượt cùng test có sai vì cùng một lý do hay không. Hai chương trình có thể in cùng một số sai: một do vòng lặp dừng sớm một phần tử, một do khởi tạo sai biến tích lũy. Ngược lại, cùng một cách hiểu sai có thể làm trượt các test khác nhau tùy phần còn lại của chương trình được viết thế nào.

Bài báo này xây dựng dữ liệu đánh giá còn thiếu mà không cần gán nhãn mới, rồi dùng nó để kiểm định quy trình kinh điển. Điểm xuất phát là: lần nộp sau của chính sinh viên thường sửa được lần nộp sai. Chúng tôi chạy lại 8.607 bài C90 của một bộ dữ liệu công khai (Orvalho et al., 2024) trong môi trường cách ly, ghép mỗi bài sai với lần nộp đạt kế tiếp của cùng sinh viên, và dùng delta debugging để thu gọn khác biệt giữa hai bài thành một bản sửa tối thiểu đã kiểm chứng: tập dòng thay đổi nhỏ nhất mà khi áp vào bài sai thì bài đạt mọi test. Một bộ phân loại tắt định gán bản sửa này vào một trong mười hai loại mức chương trình, từ biên vòng lặp đến định dạng in. Như vậy nhãn dựa trên bằng chứng đã được thực thi chứ không dựa trên phán đoán, và mọi nhãn đều có thể suy lại. Vì bản sửa thật mất cân bằng và đôi khi trộn nhiều thay đổi, chúng tôi bổ sung một benchmark thứ hai gồm các mutant một thay đổi của bài đã đạt, với loại cơ chế biết trước do cách tạo.

Với hai benchmark này, chúng tôi trả lời bốn câu hỏi. Chữ ký test làm mất bao nhiêu thông tin về cơ chế? Các mô tả không cần nhãn về *cách* output lệch khỏi đáp án có khôi phục cơ chế tốt hơn ở cùng ngân sách phân cụm không? Luật *NẾU–THÌ* quy nạp bằng Thuật toán Học Quy nạp (ILA) và phiên bản chịu nhiễu ILA-2 (Tolun and Abu-Soud, 1998; Tolun et al., 1999) có tổng quát sang sinh viên mới và bài tập mới không? Và embedding mã đóng băng gom theo cơ chế hay theo tác giả và chương trình? Mọi giả thuyết, biểu diễn, trọng số và quy tắc thống kê được đăng ký trước khi đánh

giá; mọi thay đổi đều được ghi thành sửa đổi có ghi ngày.

Đóng góp của bài báo gồm:

1. chạy lại C-Pack-IPAs có thể tái lập trong môi trường cách ly, kèm audit oracle so với hệ thống chấm lịch sử (khớp 98,42% trên 28.081 verdict test) và lần chạy lại độc lập thứ hai (khớp 99,68%);
2. một benchmark dựa trên bản sửa gồm 750 bản sửa tối thiểu đã kiểm chứng của sinh viên (339 nhãn đơn cơ chế) và một benchmark kiểm soát gồm 3.320 mutant một thay đổi, dùng chung một taxonomy và một bộ phân loại;
3. OAV độ lệch thực thi, một biểu diễn không cần nhãn và không phụ thuộc bài của các sai lệch output, cùng protocol đánh giá có trần khả phân biệt và phép so với phân hoạch ngẫu nhiên cùng độ mịn;
4. bằng chứng đăng ký trước rằng chữ ký test trộn lẫn cơ chế (từ chúng chỉ phân biệt được tối đa 73,9% bài thật và 50,7% bài tiềm lỗi), rằng bằng chứng độ lệch output khôi phục cơ chế tốt hơn trên dữ liệu kiểm soát nhưng chưa đáng tin cậy trên bản sửa thật, rằng luật ILA-2 trên các thuộc tính này chuyển được sang bài tập mới, và rằng embedding mã đóng bằng gom chương trình theo nguồn gốc.

Chúng tôi công bố runner replay, các benchmark, protocol và mã phân tích. Trong toàn bài, một nhãn mô tả điều đã thay đổi trong chương trình; nó không khẳng định sinh viên đã tin điều gì.

2. Công trình liên quan

Quan niệm sai lầm và lỗi logic của người mới học lập trình.. Các tổng quan về lập trình nhập môn mô tả những quan niệm sai về biến, luồng điều khiển, truyền tham số và “cỗ máy khái niệm” thực thi chương trình (Sorva, 2013; Qian and Lehman, 2017). Nghiên cứu về lỗi phân biệt sơ suất với lỗi xuất phát từ mô hình sai về ngôn ngữ hoặc đề bài (Altadmri and Brown, 2015; Hermans, 2021). Ettles et al. (2018) phân loại lỗi logic của sinh viên CS1 thành lỗi thuật toán, hiểu sai đề và quan niệm sai, và thấy quan niệm sai là loại phổ biến nhất và khó sửa nhất. Phản hồi dựa trên quan niệm sai có thể giúp sinh viên khi quan niệm sai đã biết trước (Gusukuma et al., 2018), còn các công cụ tương tác yêu cầu sinh viên dự đoán hành vi chương trình có thể khơi ra quan niệm sai trực tiếp (Lu and Krishnamurthi, 2024). Các nghiên cứu này thúc đẩy việc gom nhóm lỗi tự động, nhưng cũng cho thấy chỉ riêng chương trình hiếm khi tiết lộ niềm tin nào đã tạo ra nó; vì vậy chúng tôi đánh giá cơ chế mức chương trình và để các khẳng định về nhận thức ra ngoài phạm vi.

Gom cụm chương trình của sinh viên.. OverCode gom lời giải đúng theo mã đã chuẩn hóa (Glassman et al., 2015), còn Clara gom lời giải đúng theo tương đương động để sửa bài sai (Gulwani et al., 2018). Với bài sai, Kaleeswaran et al. (2016) gom theo chiến lược sửa để giảng viên chỉ sửa một đại diện mỗi cụm; MistakeBrowser gom bài sai theo phép biến đổi mã mà sinh viên đã áp dụng khi tự sửa (Head et al., 2017); TraceDiff đối chiếu vết thực thi của chương trình lỗi với bản sửa gần nhất (Suzuki et al., 2017). Embedding chương trình học từ mã và thực thi đã được dùng để lan truyền phản hồi (Piech et al., 2015), và Shi et al. (2021) gom embedding của bài sai dạng khối để giúp chuyên gia phát hiện quan niệm sai. Với C, InvAASTCluster kết hợp bất biến động với cây cú pháp trừu tượng hóa (Orvalho et al., 2025), còn MENTOR ghép phân cụm, định vị lỗi bằng công thức và sửa lỗi bằng LLM (Orvalho et al., 2026). Các hệ thống này dùng bản sửa hoặc lời giải đúng như một phần của phương pháp. Chúng tôi chỉ dùng bản sửa về sau của chính sinh viên để xây dựng nhãn đánh giá, và hỏi các biểu diễn có sẵn trước khi có bản sửa khôi phục nhãn đó tốt đến đâu.

Khai phá quan niệm sai bằng mô hình học.. Các mô hình attention trên cây con định vị lỗi logic chỉ từ nhãn đúng/sai (Hoq et al., 2025), thành phần tri thức theo mẫu được trích từ mã sinh viên bằng học biểu diễn và phân cụm (Hoq et al., 2026), và LLM phát hiện quan niệm sai từ tập mẫu mã trong McMining, với benchmark tìm quan niệm sai đã biết vào mã Python (Al-Hossami and Bunesu, 2026). Các công cụ lớp học gần đây gom lỗi do LLM nhận diện cho giảng viên (Liding et al., 2026), định hướng dạy kèm theo các quan niệm sai đã biết (Niousha et al., 2026), hoặc tinh chỉnh tín hiệu phát hiện tự động bằng đánh giá của chuyên gia (Aruleba et al., 2026). PyMETA cho thấy độ chính xác chẩn đoán phụ thuộc mạnh vào cách định nghĩa nhãn chuẩn: mô hình gán tốt lỗi thực thi đầu tiên lại khớp kém hơn nhiều với lộ trình sửa của chuyên gia (Li et al., 2026). Nghiên cứu của chúng tôi cùng mối quan tâm đó và làm cho định nghĩa nhãn trở nên tường minh, kiểm chứng được và tái lập được.

Benchmark, mutant và bản sửa.. C-Pack-IPAs cung cấp bài C90 của 25 bài tập kèm mã sinh viên ẩn danh, test và log chấm lịch sử (Orvalho et al., 2024); ITSP là benchmark trước đó gồm chương trình C lỗi và đã sửa (Yi et al., 2017). Phân tích đột biến tìm lỗi có kiểm soát; trong kiểm thử phần mềm, khả năng phát hiện mutant tương quan với khả năng phát hiện lỗi thật (Just et al., 2014), còn mutant khái niệm xây từ ví dụ sai của sinh viên làm lộ quan niệm sai về đặc tả (Prasad et al., 2024). Delta debugging cô lập khác biệt tối thiểu gây lỗi giữa một cấu hình lỗi và một cấu hình đạt (Zeller and Hildebrandt, 2002). Chúng tôi kết hợp các ý tưởng này: chạy lại corpus, thu

gọn bản sửa của chính sinh viên thành thay đổi tối thiểu đã kiểm chứng, và ghép các nhãn này với mutant một cơ chế có kiểm soát.

Quy nạp luật.. Thuật toán Học Quy nạp (ILA) trích luật sản xuất theo từng lớp từ các ví dụ thuộc tính-giá trị (Tolun and Abu-Soud, 1998); ILA-2 thêm hệ số phạt để chịu được dữ liệu nhiễu (Tolun et al., 1999). ILA đôi khi được nêu cùng các phương pháp phân cụm, nhưng bản chất là quy nạp luật có giám sát, và chúng tôi dùng đúng như vậy: để phát biểu luật NẾU-THÌ ảnh xạ bằng chứng thực thi sang cơ chế, và đo xem các luật đó còn đúng tới đâu trên những sinh viên không tham gia quy nạp.

3. Dữ liệu và xây dựng benchmark

3.1. Corpus và chạy lại trong môi trường cách ly

Chúng tôi dùng các bài nộp gốc của C-Pack-IPAs bản phát hành 26 (Orvalho et al., 2024): 8.607 chương trình C90 của 246 sinh viên ẩn danh cho 25 bài tập thuộc ba buổi thực hành, với 106 test đầu vào/đầu ra. Log lịch sử ghi verdict từng test nhưng thường không lưu output của test đạt, và 9.425 ô test chưa từng được chạy hoặc không được ghi log. Để có đủ bằng chứng thực thi, chúng tôi chạy lại mọi chương trình không rõ ràng trong một container chỉ dùng cho nghiên cứu: không mạng, hệ thống tệp gốc chỉ đọc, bỏ mọi quyền Linux trừ các quyền cần để chuyển sang một người dùng không đặc quyền riêng cho mỗi lượt chạy, với giới hạn 3 giây CPU, 5 giây thời gian thực và 512 MiB bộ nhớ cho mỗi test. Chương trình được biên dịch bằng gcc 12.2.0 với cờ của khóa học (`-ansi -pedantic -Wall -Wextra`); vì khóa học còn dùng `-Werror`, chúng tôi ghi lại việc biên dịch có sạch cảnh báo hay không thay vì loại chương trình. Container chỉ trả về các byte output thô; oracle nằm ở máy chủ, nơi quyết định mọi verdict.

Lần chạy lại khớp 98,42% trong 28.081 verdict test lịch sử theo chính sách khớp chính xác và mã thoát 0 (được chọn trên partition huấn luyện trong bốn chính sách so sánh; Bảng 1), tái tạo byte-đúng-byte output đã ghi của 96,1% các câu trả lời sai lịch sử, và khớp kết quả biên dịch lịch sử theo ngữ nghĩa `-Werror` với 99,76% chương trình. Một lần chạy lại độc lập thứ hai từ commit sạch khớp 99,68% ô test; 56 chương trình có kết quả khác nhau giữa hai lần (nhiều khả năng do hành vi không xác định) bị loại khỏi mọi item (sửa đổi A5).

3.2. Chia dữ liệu

Bài nộp được chia trước khi có bất kỳ nhãn nào thành các partition huấn luyện, kiểm định và kiểm tra (70/15/15% số thành phần liên thông), sao cho mọi bài của một sinh viên, xuyên năm và xuyên bài tập, cùng mọi mã

Bảng 1: Tỷ lệ khớp (%) giữa lần chạy lại cách ly và verdict lịch sử từng test theo bốn chính sách so sánh output. Chính sách được chọn trên partition huấn luyện trước khi có bất kỳ nhãn cơ chế nào.

Chính sách so sánh	Huấn luyện	Kiểm định	Kiểm tra
Khớp chính xác, mã thoát 0 (được chọn)	98,23	99,04	98,52
Khớp chính xác, mã thoát bất kỳ	98,16	98,70	98,36
Bỏ qua khoảng trắng cuối	89,03	89,75	91,13
So theo token cách bởi khoảng trắng	83,88	86,22	86,47

nguồn trùng byte đều nằm trong một partition. Mọi item suy ra đều thừa kế partition của bài nộp gốc.

3.3. Nhãn dựa trên bản sửa

Với mỗi bài nộp sai (ít nhất một test không đạt), chúng tôi tìm lần nộp kế tiếp của cùng sinh viên, cùng bài tập, cùng năm mà đạt mọi test. Với mỗi sự kiện sửa như vậy, chúng tôi giữ lần nộp sai cuối cùng trước nó (sửa đổi A2), được 750 cặp sai-đã sửa. Khác biệt theo dòng giữa hai chương trình, bỏ qua thụt lề, được chia thành các hunk. Delta debugging theo hunk (Zeller and Hildebrandt, 2002) tìm một tập con 1-tối thiểu các hunk mà khi áp vào bài sai thì bài đạt mọi test; mọi ứng viên đều được chạy trong runner cách ly (ngân sách 64 lần thử mỗi cặp). Cả hai đầu mút được kiểm lại trong cùng runner trước khi tìm. Thay đổi tối thiểu đã kiểm chứng được một bộ phân loại tất định xếp vào một trong mười hai loại mức chương trình (Bảng 2); bộ phân loại làm việc trên từng hunk đã kiểm chứng: câu lệnh được chèn hoặc xóa nguyên vẹn được nhận ra sau khi trượt đoạn token về biên câu lệnh, các dòng giống hệt bị xóa ở chỗ này và chèn ở chỗ khác là thay đổi vị trí, còn mọi token thay đổi khác được quy về ngữ cảnh cú pháp gần nhất có vai trò xác định (đầu vòng lặp, điều kiện rẽ nhánh, giá trị khởi tạo, định dạng in, ...). Một cặp chỉ nhận nhãn đơn cơ chế khi bản sửa tối thiểu thuộc đúng một loại khác OTHER; bản sửa trải nhiều loại được giữ là MULTI và không vào tập nhãn chuẩn chính. Bài sửa, bản vá và nhãn không bao giờ được dùng làm đặc trưng.

3.4. Tiêm cơ chế có kiểm soát

Bản sửa thật có giá trị sinh thái nhưng nhiều, mất cân bằng và phụ thuộc điều sinh viên tình cờ làm. Vì vậy chúng tôi xây thêm một benchmark kiểm soát. Từ các chương trình đã đạt và sạch cảnh báo (tối đa 40 bài mỗi bài tập, mỗi sinh viên và năm một bài), chúng tôi sinh mutant một thay đổi bằng các toán tử khớp với cùng taxonomy: độ chặt của phép so sánh trong vòng lặp hoặc điều kiện, phép nối logic, điểm bắt đầu vòng lặp, giá trị khởi

Bảng 2: Taxonomy cơ chế mức chương trình. Một loại mô tả điều mà bản sửa tối thiểu đã kiểm chứng (hoặc thay đổi được tiêm) thay đổi; cột phải là giả thuyết quan niệm sai để giảng viên kiểm tra, không phải chẩn đoán.

Loại	Thay đổi tối thiểu sửa...	Giả thuyết cần kiểm tra
Biên vòng lặp	điều kiện, điểm bắt đầu hoặc bước nhảy của vòng lặp	lệch một, chỉ số từ 0 hay 1
Điều kiện rẽ nhánh	điều kiện hoặc cấu trúc if/switch/else	biên so sánh, <code>&&</code> và <code> </code>
Khởi tạo	giá trị khởi tạo khi khai báo hoặc gán hằng ngoài vòng lặp	khởi tạo biến tích lũy, giá trị lớn nhất
Biểu thức tính	toán tử, toán hạng trong phép gán, return, biểu thức được in	công thức, độ ưu tiên toán tử
Kiểu số	kiểu, ép kiểu, hằng thực, hàm toán học	chia nguyên
Định dạng in Văn bản in	đặc tả chuyển đổi của printf chữ trong output hoặc lệnh chỉ in hằng	độ chính xác, độ rộng đọc đặc tả output
Đọc dữ liệu	lời gọi scanf/getchar	định dạng đầu vào, EOF
Vị trí câu lệnh	câu lệnh hoặc dấu ngoặc bị di chuyển	phạm vi khối, đặt lại biến trong vòng lặp
Thiếu bước	chèn nguyên câu lệnh	thiếu cập nhật hoặc trường hợp
Thừa bước	xóa nguyên câu lệnh	cập nhật hai lần, in gỡ lỗi
Luồng điều khiển	break/continue/return	thoát sớm

tạo hằng, toán tử số học, bỏ ép kiểu thực hoặc hằng thực, độ chính xác của printf, bỏ ký tự xuống dòng cuối, chữ hoa/thường của văn bản in, xóa câu lệnh, lặp câu lệnh, và chuyển câu lệnh vào trong vòng lặp. Mỗi loại và mỗi bài gốc được đột biến ở một vị trí ngẫu nhiên với seed cố định. Mutant chỉ được giữ nếu biên dịch sạch cảnh báo và trượt ít nhất một test, tức là có thể đã là một bài nộp sai thật của khóa học. Một unit test kiểm tra rằng bộ phân loại ở Mục 3.3 xếp phép sửa ngược của mọi toán tử vào đúng loại của toán tử, nên hai bộ nhãn dùng chung một định nghĩa.

4. Phương pháp

4.1. Biểu diễn đối tượng-thuộc tính-giá trị

Mỗi chương trình sai (đối tượng) được mô tả bằng các thuộc tính phân loại. Mọi biểu diễn chỉ được tính từ chương trình sai và các lượt chạy lại của chính nó. Từ vựng được khớp, không dùng nhãn, trên các bài sai thuộc

partition huấn luyện của cùng bài tập; giá trị chưa gặp khi huấn luyện được ánh xạ thành *chưa biết*.

- **Kết quả test** (OAV theo test case của cách phát biểu kinh điển): mỗi test một thuộc tính với giá trị đạt, sai, lỗi chạy hoặc quá thời gian.
- **Cấu trúc**: sự hiện diện của 16 cấu trúc C (các loại vòng lặp, so sánh chặt/không chặt, chỉ số mảng, con trỏ, ...).
- **Kết quả + stdout** và **Kết hợp + stdout**: các biểu diễn mạnh nhất của quy trình trước đây của chúng tôi, thêm quan hệ output theo từng test (khớp, khác khoảng trắng, rỗng, trùng đáp án test khác, khác) và dải khoảng cách sửa; bản kết hợp còn có khối cấu trúc.
- **Độ lệch** (đề xuất): khối kết quả test cộng, với mỗi test, chín mô tả không cần nhãn về *cách* output lệch khỏi đáp án—số dòng, ký tự xuống dòng cuối, chỉ khác khoảng trắng, quan hệ giữa các số tương ứng (lệch một, bị cắt phần thập phân, độ chính xác, dấu, bằng không, tỷ lệ, chỉ khác cách viết), quan hệ giữa các từ (chỉ khác hoa/thường, lỗi chính tả, thiếu hoặc thừa từ), vị trí dòng khác đầu tiên, quan hệ tiền tố và việc trùng đáp án của test khác—cùng các mô tả đó gộp trên các test sai, vốn không phụ thuộc bài.
- **Bằng chứng** (đề xuất): độ lệch cộng mười tám manh mối mã tĩnh (so sánh chặt và không chặt trong vòng lặp, vòng lặp bắt đầu khác 0, giá trị khởi tạo hằng, phép chia không có toán hạng thực, ép kiểu, độ chính xác trong printf, lệnh in trong vòng lặp, chữ in ra không có trong đáp án nào, ...).
- **Embedding**: embedding mã đóng băng của mã nguồn thô (Qwen3-Embedding-0.6B, Zhang et al. 2025), chuẩn hóa L2.

Trọng số các khối được cố định trong protocol đăng ký trước mọi đánh giá và không được tinh chỉnh (kết quả test 1; kết quả + stdout 0,6/0,4; kết hợp + stdout 0,48/0,12/0,4; độ lệch 0,3 test, 0,4 độ lệch từng test, 0,3 gộp; bằng chứng 0,2/0,3/0,2 và 0,3 manh mối mã).

4.2. Phân cụm và luật

Các chương trình sai của mỗi bài tập tạo thành một cohort và được phân cụm theo kiểu chuyển nạp. K-means chạy trên mã hóa one-hot được nhân với căn bậc hai trọng số thuộc tính (seed 7, 42, 91), còn phân cụm phân cấp dùng liên kết trung bình trên khoảng cách Hamming có trọng số; embedding dùng K-means Euclid và khoảng cách cosine. Phân tích chính cho mọi phương

pháp số cụm bằng số loại nhãn có trong cohort (k theo nhãn, giới hạn bởi số mẫu biểu diễn khác nhau), nhằm tách biểu diễn khỏi việc chọn mô hình; phân tích phụ chọn $k \in [2, 10]$ theo silhouette mà không dùng nhãn. Baseline rẻ nhất gom các chữ ký test giống hệt.

Để có luật NẾU–THÌ, chúng tôi dùng ILA (Tolun and Abu-Soud, 1998) và ILA-2 (Tolun et al., 1999) trên các thuộc tính không phụ thuộc bài: các mô tả độ lệch gộp và mạnh mỗi mã (*bằng chứng*), hoặc chỉ trạng thái kết quả gộp và tỷ lệ test sai (*tóm tắt kết quả*). Luật được quy nạp từ nhãn của partition huấn luyện, áp dụng theo thứ tự độ chính xác, và ví dụ không khớp luật nào được tính là từ chối trả lời và là lỗi. Hệ số phạt của ILA-2 là đại lượng duy nhất được tinh chỉnh, trên partition kiểm định, trong tập $\{1, 2, 5\}$. Chúng tôi so với cây CART độ sâu 4 trên cùng thuộc tính và với lớp đa số, trên bài đã thấy với sinh viên chưa thấy, và trên bài mới (buổi thực hành 4) với sinh viên chưa thấy.

4.3. Độ đo và giả thuyết đăng ký trước

Độ đo chính là chỉ số Rand hiệu chỉnh (ARI; Hubert and Arabie 1985) giữa các cụm của một cohort và loại nhãn chuẩn, lấy trung bình trên các cohort có ít nhất sáu item và hai loại; chúng tôi cũng báo cáo thông tin tương hỗ hiệu chỉnh (Vinh et al., 2010), F1 theo cặp và độ thuần. *Trần khả phân biệt* của một biểu diễn là $\sum_b \max_y n_{b,y} / N$ trên các nhóm b gồm những vector thuộc tính giống hệt: không phép phân cụm hay bộ phân loại nào chỉ đọc biểu diễn đó có thể vượt qua trần này. Vì OAV độ lệch mịn hơn OAV kết quả test, trần của nó không thể thấp hơn; do đó chúng tôi so nó với trần kỳ vọng của một phân hoạch ngẫu nhiên chia mỗi nhóm kết quả test thành các nhóm con có cùng kích thước (200 hoán vị).

Protocol, đăng ký trước mọi đánh giá cơ chế, nêu năm giả thuyết kèm tiêu chí bác bỏ (bảng sửa đổi có ghi ngày được ghi trước lần chạy kiểm tra niêm phong, sáu trong số đó trước mọi kết quả đánh giá; Phụ lục Phụ lục A): H1a, chữ ký test giống hệt trộn lẫn cơ chế (cận trên 95% của trần trung bình theo kết quả test dưới 0,95); H1b, phần phân hoạch mịn thêm của OAV độ lệch mang thông tin cơ chế vượt mức độ mịn; H2, OAV độ lệch cho ARI cao hơn OAV kết quả test; H3, OAV bằng chứng cho ARI cao hơn baseline mạnh nhất trước đây là kết hợp + stdout; H4, luật ILA-2 trên bằng chứng không phụ thuộc bài đạt macro-F1 kiểm tra cao hơn luật trên tóm tắt kết quả, ở benchmark thật; H5, ở benchmark tiêm lỗi, cụm của embedding mã đóng băng khớp với chương trình gốc nhiều hơn là với cơ chế được tiêm. Chênh lệch được tóm tắt bằng trung bình trên các cohort kèm khoảng bootstrap 95% (10.000 lần lấy mẫu) và kiểm định dấu hạng Wilcoxon có hiệu chỉnh Holm; so sánh luật dùng bootstrap theo sinh viên. Một giả thuyết chỉ được ủng hộ khi khoảng của chênh lệch loại trừ 0 theo đúng chiều trên mọi benchmark

mà nó nêu. Vì item đơn cơ chế thật rất thưa trong partition kiểm tra niêm phong (0 cohort đủ điều kiện), sửa đổi A3 đã đăng ký đánh giá phân cụm trên benchmark thật bằng các cohort đầy đủ theo bài tập (phân cụm không khớp gì trên nhân); benchmark tiềm lỗi được đánh giá trên các cohort của partition kiểm tra, kết quả trên cohort đầy đủ được công bố cùng dữ liệu. Quy nạp luật luôn học trên partition huấn luyện và được chấm trên partition kiểm tra.

5. Kết quả

5.1. Các benchmark

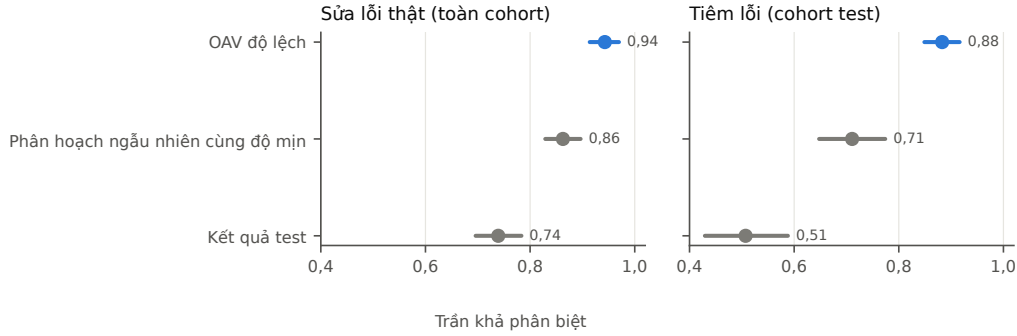
Trong 750 sự kiện sửa, 746 qua được kiểm tra hai đầu mút và 743 đạt bản sửa 1-tối thiểu trong ngân sách thử; 72% bản sửa tối thiểu chỉ thay đổi một hoặc hai hunk. 54,4% bản sửa tối thiểu trải trên nhiều loại và bị loại khỏi tập nhân chuẩn; 339 item đơn cơ chế còn lại chủ yếu thuộc loại văn bản in (khóa học chấm output khớp từng byte), tiếp theo là biểu thức tính, thiếu bước, điều kiện rẽ nhánh, biên vòng lặp, định dạng in và luồng điều khiển (7 trong 8 bản sửa luồng điều khiển được audit là thêm `return 0` mà C90 cần để có mã thoát 0). Benchmark kiểm soát gồm 3.320 mutant thuộc chín loại; 16,6% mutant ứng viên vẫn đạt mọi test và bị loại vì tương đương trên bộ test. Số liệu đầy đủ được công bố cùng benchmark (`stats.json`).

5.2. Chữ ký test trộn lẫn cơ chế (H1a, H1b)

Các chữ ký test giống hệt thường che giấu những cơ chế khác nhau (Hình 1). Tốt nhất, một hàm bất kỳ của OAV kết quả test chỉ gán đúng cơ chế cho 73,9% bài thật (trần trung bình 0,74, KTC 95% [0,70; 0,78]) và 50,7% bài tiềm lỗi ([0,43; 0,59]), nên H1a được ủng hộ trên cả hai benchmark. Nói cách khác, 35,5% trong 2.087 cặp bài thật có cùng chữ ký test trong một bài tập là khác cơ chế, và tỷ lệ này là 81,6% trong 2.901 cặp tiềm lỗi. Phần mịn thêm của OAV độ lệch nâng trần thêm 0,080 [0,052; 0,112] trên bài thật và 0,172 [0,121; 0,224] trên bài tiềm lỗi so với một phân hoạch ngẫu nhiên cùng độ mịn (H1b được ủng hộ; Holm $p = 0,0018$ và 0,0000). Vậy các phân biệt thêm vào mang thông tin cơ chế chứ không chỉ tăng độ phân giải.

5.3. Phân cụm theo cơ chế (H2, H3)

Bảng 3 and Hình 2 báo cáo mức khôi phục cơ chế ở cùng ngân sách phân cụm. Trên benchmark kiểm soát, cụm của OAV kết quả test hầu như không khớp với cơ chế (ARI 0,09, chữ ký trùng khớp 0,10), trong khi OAV độ lệch đạt 0,33: chênh lệch 0,235 [0,143; 0,331] với K-means và 0,214 [0,117; 0,314] với HAC, cao hơn ở 16/20 cohort. Trên bản sửa thật, OAV kết quả test đã đạt 0,29 vì loại văn bản in chiếm đa số tạo ra chữ ký đặc trưng; OAV độ



Hình 1: Trần khả phân biệt: tỷ lệ cao nhất số item mà một hàm bất kỳ của biểu diễn có thể gán đúng cơ chế. OAV độ lệch vượt cả OAV kết quả test lẫn một phân hoạch ngẫu nhiên có cùng kích thước nhóm. Điểm là trung bình trên các cohort; thanh là khoảng bootstrap 95%.

lệch đạt 0,35 với K-means và 0,38 với HAC, nhưng các chênh lệch (0,054, KTC $[-0,081; 0,182]$; 0,086, KTC $[-0,025; 0,200]$) không đáng tin cậy. Do đó H2 chỉ được ủng hộ trên benchmark kiểm soát và, theo quy tắc đã đăng ký, không được ủng hộ về tổng thể.

H3 không được ủng hộ: OAV bằng chứng và baseline kết hợp + stdout trước đây không phân biệt được trên cả hai benchmark (chênh lệch $-0,006$ và $0,002$). Các quan hệ output đơn giản đã nắm gần hết những gì phân cụm theo khoảng cách có thể khai thác; kết quả + stdout đạt 0,36 và 0,34. Chọn k bằng silhouette thay cho số loại nhãn làm mọi điểm số giảm nhẹ và giữ nguyên thứ tự (Bảng 3).

5.4. *Embedding mã đóng bằng gom chương trình, không gom cơ chế (H5)*

Cụm Qwen3-Embedding của các mutant khớp với chương trình gốc sinh ra mutant nhiều hơn là với cơ chế được tiêm, chênh 0,834 ARI (KTC 95% $[0,695; 0,958]$; H5 được ủng hộ). ARI theo cơ chế của chúng là $-0,11$ trên bài tiêm lỗi và 0,06 trên bài thật. Những thay đổi một token quyết định hành vi chương trình hầu như không dịch chuyển một embedding đa dụng, trong khi dấu ấn tác giả và phong cách lời giải chi phối nó.

5.5. *Luật NẾU-THÌ (H4)*

Luật ILA-2 trên các thuộc tính bằng chứng không phụ thuộc bài dự đoán cơ chế của item thật thuộc partition kiểm tra với macro-F1 0,26 (độ phủ 0,69, độ chính xác trên item có trả lời 0,70), so với 0,13 của luật trên tóm tắt kết quả và 0,11 của lớp đa số (Bảng 5). Chênh lệch 0,094 $[-0,011; 0,171]$ chỉ dựa trên 29 item đơn cơ chế của sinh viên kiểm tra và chứa 0, nên H4

Bảng 3: Khôi phục cơ chế trên hai benchmark: ARI trung bình trên các cohort (k theo nhãn nếu không ghi khác) và trần khả phân biệt trung bình. Bản sửa thật: cohort đầy đủ theo bài tập (sửa đổi A3); mutant tiềm lỗi: cohort của partition kiểm tra niêm phong.

Biểu diễn	K-means	HAC	K-means, k silhouette	Trần
<i>Bản sửa thật</i>				
Kết quả test (OAV kinh điển)	0,292	0,292	0,239	0,739
Kết quả + stdout	0,363	0,367	0,313	0,907
Kết hợp + stdout	0,306	0,334	0,314	0,980
Hiện diện AST	0,064	0,029	0,080	0,772
Manh mối mã	0,141	0,157	0,105	0,931
Embedding mã	0,060	0,077	0,063	–
OAV độ lệch (đề xuất)	0,345	0,378	0,295	0,943
OAV bằng chứng (đề xuất)	0,300	0,324	0,279	0,986
Chữ ký test trùng khớp	0,281	–	–	–
<i>Mutant tiềm lỗi</i>				
Kết quả test (OAV kinh điển)	0,093	0,084	0,088	0,507
Kết quả + stdout	0,338	0,299	0,332	0,839
Kết hợp + stdout	0,335	0,293	0,309	0,914
Hiện diện AST	–0,066	–0,068	–0,052	0,353
Manh mối mã	0,009	–0,009	–0,020	0,654
Embedding mã	–0,115	–0,104	–0,114	–
OAV độ lệch (đề xuất)	0,328	0,297	0,328	0,883
OAV bằng chứng (đề xuất)	0,337	0,283	0,227	0,964
Chữ ký test trùng khớp	0,096	–	–	–

không được ủng hộ. Trên benchmark tiềm lỗi, nơi mọi loại đều có mặt, luật bằng chứng đạt macro-F1 0,65 trên bài đã thấy và 0,35 trên bài mới, trong khi luật tóm tắt kết quả đạt 0,03 và 0,04. Hệ số phạt chịu nhiễu của ILA-2 đổi một chút độ chính xác lấy nhiều độ phủ: ILA gốc trả lời 0,34 item kiểm tra tiềm lỗi với độ chính xác 0,94, còn ILA-2 trả lời 0,81 với độ chính xác 0,74. Bảng 6 liệt kê các luật có độ chính xác cao nhất học từ bản sửa thật; chúng đọc giống luật sản xuất của cách phát biểu kinh điển, nhưng kết luận là cơ chế mức chương trình và độ chính xác được đo trên những sinh viên không tham gia quy nạp.

5.6. Cơ chế nào được lợi (khám phá)

Theo từng loại (Phụ lục Phụ lục D), trên bản sửa thật chữ ký test nhận ra tốt văn bản in (0,98), biên vòng lặp (0,88) và luồng điều khiển (0,83), nhưng không nhận ra định dạng in (0,00), điều kiện rẽ nhánh (0,24) hay thiếu bước (0,37); với OAV độ lệch các giá trị này tăng lên 0,69, 0,59 và 0,56.

Bảng 4: Các giả thuyết đã đăng ký: chênh lệch trung bình trên các cohort, khoảng bootstrap 95% và kết luận (H1a báo cáo chính trần trung bình theo kết quả test).

Giả thuyết	Bản sửa thật			Mutant tiềm lỗi		
	TB	KTC 95%	Kết luận	TB	KTC 95%	Kết luận
H1a	0,739	[0,696; 0,783]	ủng hộ	0,507	[0,429; 0,588]	ủng hộ
H1b	0,080	[0,052; 0,112]	ủng hộ	0,172	[0,121; 0,224]	ủng hộ
H2 kmeans	0,054	[−0,081; 0,182]	không ủng hộ	0,235	[0,143; 0,331]	ủng hộ
H2 hac	0,086	[−0,025; 0,200]	không ủng hộ	0,214	[0,117; 0,314]	ủng hộ
H3 kmeans	−0,006	[−0,097; 0,117]	không ủng hộ	0,002	[−0,055; 0,067]	không ủng hộ
H3 hac	−0,010	[−0,111; 0,119]	không ủng hộ	−0,010	[−0,060; 0,041]	không ủng hộ
H5	−	−	−	0,834	[0,695; 0,958]	ủng hộ

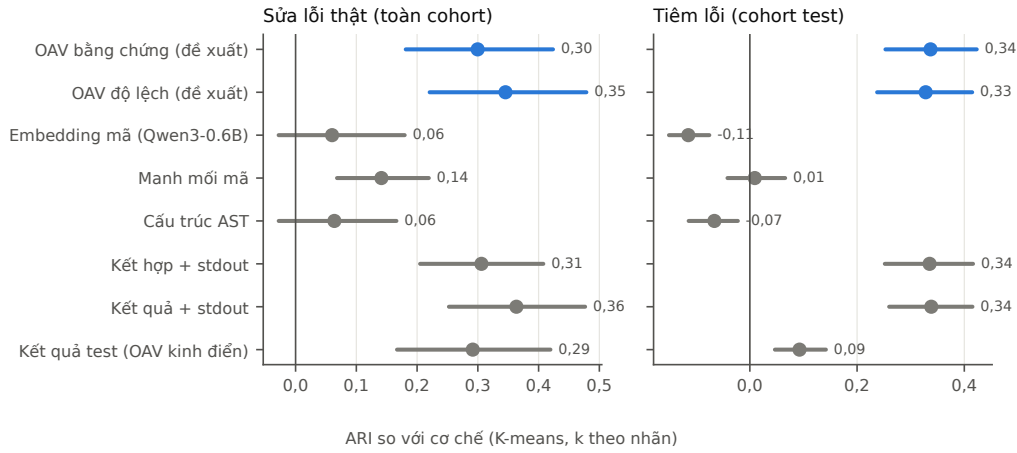
Bảng 5: Luật NẾU–THÌ dự đoán cơ chế của sinh viên thuộc partition kiểm tra từ các thuộc tính không phụ thuộc bài, trên bản sửa thật. Từ chối trả lời được tính là lỗi trong macro-F1; độ chính xác chọn lọc đo trên các item có trả lời.

Hướng đánh giá	Mô hình (thuộc tính)	Macro-F1	Độ phủ	Độ chính xác chọn lọc
Bài đã thấy	ILA (bằng chứng)	0,254	0,586	0,765
Bài đã thấy	ILA-2 (bằng chứng)	0,256	0,690	0,700
Bài đã thấy	CART (bằng chứng)	0,244	1,000	0,621
Bài đã thấy	ILA (tóm tắt kết quả)	0,000	0,000	0,000
Bài đã thấy	ILA-2 (tóm tắt kết quả)	0,130	0,828	0,583
Bài đã thấy	CART (tóm tắt kết quả)	0,251	1,000	0,586
Bài đã thấy	Lớp đa số	0,109	1,000	0,483
Bài mới	ILA (bằng chứng)	0,354	1,000	0,500
Bài mới	ILA-2 (bằng chứng)	0,354	1,000	0,500
Bài mới	CART (bằng chứng)	0,381	1,000	0,500
Bài mới	ILA (tóm tắt kết quả)	0,000	0,000	0,000
Bài mới	ILA-2 (tóm tắt kết quả)	0,214	0,750	0,500
Bài mới	CART (tóm tắt kết quả)	0,214	1,000	0,375
Bài mới	Lớp đa số	0,136	1,000	0,375

Tính trung bình theo loại, mức khôi phục tăng từ 0,54 lên 0,71 trên bài thật và từ 0,50 lên 0,72 trên bài tiềm lỗi. Vậy lợi ích của bằng chứng output tập trung ở các cơ chế thiếu số mà ARI cấp cohort ít coi trọng, điều giải thích vì sao chênh lệch ARI trên benchmark thật nhỏ.

6. Thảo luận

Chữ ký test nói được gì và không nói được gì với giảng viên.. Quy trình kinh điển—OAV theo test case, phân cụm K-means hoặc Hamming, luật NẾU–THÌ quy nạp—nhận ra tốt lỗi phổ biến nhất của khóa học này: lỗi văn bản in tạo chữ ký đặc trưng, và trên bản sửa thật, cụm theo chữ ký đạt ARI trung bình 0,29. Nhưng biểu diễn này không tách được các cơ chế làm trượt



Hình 2: Chỉ số Rand hiệu chỉnh giữa cụm và cơ chế (K-means, k theo nhãn). Xanh: biểu diễn đề xuất; xám: baseline. Điểm là trung bình trên các cohort; thanh là khoảng bootstrap 95% trên các cohort.

cùng những test. Trên các mutant có kiểm soát của cùng các chương trình, nơi mọi cơ chế đều có mặt, cụm của nó hầu như không khớp với cơ chế (ARI 0,09), và ngay cả một bộ phân loại hoàn hảo chỉ đọc chữ ký cũng chỉ gán đúng tối đa 50,7% item. Một dashboard dựng trên chữ ký test vì vậy báo cáo chính xác các lỗi phổ biến, thường ở mức đặc tả, và gộp lẫn các lỗi khái niệm ít gặp hơn—điều kiện rẽ nhánh sai, thiếu bước, định dạng in—vốn chính là lý do để phân cụm ngay từ đầu.

Bảng chứng output là cải tiến rẻ nhất, không phải mô hình mới.. Mọi thứ giúp khôi phục cơ chế tốt hơn đều đã có sẵn trong output của hệ thống chấm: quan hệ giữa output thực và output mong đợi, và trong OAV độ lệch là kiểu sai lệch số và chữ. Trên dữ liệu kiểm soát, điều này làm ARI tăng hơn ba lần ở cùng ngân sách phân cụm, và trên bản sửa thật nó nâng mức khôi phục theo loại của các cơ chế thiếu số. Các mô tả phong phú hơn của chúng tôi không vượt quan hệ output đơn giản khi phân cụm (H3), nên chúng tôi không khẳng định có phương pháp phân cụm tốt hơn. Giá trị của chúng nằm ở chỗ khác: chúng không phụ thuộc bài, nên luật học trên một số bài tập áp dụng được cho bài khác. Luật trên các mô tả độ lệch gộp giữ macro-F1 0,35 trên bài tập mới của benchmark kiểm soát, trong khi luật trên tóm tắt kết quả gần như bằng 0; còn chữ ký theo từng test thậm chí không thể so sánh giữa các bài tập.

Luật như những giả thuyết đã kiểm chứng.. ILA và ILA-2 tạo ra đúng loại luật sản xuất mà cách phát biểu kinh điển yêu cầu, và hệ số phạt của ILA-2

Bảng 6: Các luật ILA-2 chính xác nhất quy nạp từ bản sửa thật thuộc partition huấn luyện (hỗ trợ: số item huấn luyện đúng/khớp).

NẾU	THÌ cơ chế	Hỗ trợ
chương trình không có lệnh in nào	thiếu bước xử lý	10/10
chương trình không có lệnh đọc dữ liệu	thiếu bước xử lý	8/8
các số in ra bằng đáp án nhưng viết khác cách in	đặc tả định dạng	8/8
ký tự xuống dòng cuối đúng như đáp án và in ít số hơn đáp án	văn bản in ra	7/7
các test sai đều quá thời gian	biên vòng lặp	4/4
output là phần đầu bị cắt của đáp án	văn bản in ra	78/79
output chỉ khác đáp án ở khoảng trắng	văn bản in ra	97/107
output có chữ mà đáp án không có	văn bản in ra	20/24

đổi một chút độ chính xác lấy độ phủ lớn hơn nhiều. Chúng tôi trình bày kết luận của luật như một giả thuyết cơ chế kèm độ chính xác đo trên dữ liệu giữ lại, và cho hệ thống từ chối trả lời khi không luật nào khớp. Trong ứng dụng học tập của chúng tôi, mỗi cụm của một lớp giờ hiển thị giả thuyết như vậy, tỷ lệ thành viên khớp luật, một phép kiểm tra một phút để xác nhận hoặc bác bỏ với sinh viên, và một gợi ý giảng lại. Tuy nhiên, trên bản sửa thật, ưu thế của luật bằng chứng so với luật tóm tắt kết quả chưa đáng tin cậy về thống kê với 29 item kiểm tra niêm phong, nên các luật này nên được xem là điểm khởi đầu cho phán đoán của giảng viên.

Embedding tìm ra tác giả trước khi tìm ra cơ chế.. Một embedding mã đa dụng mạnh đã gom các mutant theo chương trình gốc của chúng: những thay đổi một token quyết định hành vi hầu như không dịch chuyển vector, trong khi tên biến, cách trình bày và chiến lược lời giải chi phối nó. Điều này không mâu thuẫn với các thành công trước đây của embedding được huấn luyện cho đúng nhiệm vụ (Shi et al., 2021; Piech et al., 2015); nó cho thấy một embedding đóng băng cần bằng chứng thực thi hoặc được huấn luyện nhạy với cơ chế trước khi cụm của nó có thể được đọc như các nhóm lỗi.

Đánh giá dựa trên bản sửa.. Bản sửa tối thiểu đã kiểm chứng biến 750 sự kiện sửa tự nhiên thành các nhãn có thể audit, với chi phí 7.040 lần thực thi chương trình và không tốn giờ gán nhãn thủ công. Hai phát hiện có ý nghĩa ngoài phạm vi nghiên cứu này. Thứ nhất, 54,4% bản sửa tối thiểu thay đổi nhiều hơn một loại mã, nên việc chia các bài sai thành các cụm đơn cơ chế tự thân đã là một sự giản lược; cách gom chồng lấn hoặc đa nhãn cần được quan tâm hơn. Thứ hai, một benchmark kiểm soát xây theo cùng

taxonomy làm lộ các giới hạn mà phân bố thật mất cân bằng che giấu, và hai benchmark bất đồng đúng ở chỗ loại chiếm đa số của dữ liệu thật vốn dễ nhận ra.

6.1. Hạn chế và mối đe dọa tính hợp lệ

Nhân mô tả chương trình, không mô tả tư duy.. Một nhân dựa trên bản sửa ghi lại điều sinh viên đã thay đổi để chương trình đạt; cùng một thay đổi có thể đến từ một quan niệm sai, một sơ suất hay việc đọc nhầm đặc tả, và sinh viên có thể sửa triệu chứng mà không thay đổi niềm tin. Vì vậy chúng tôi đánh giá cơ chế mức chương trình và diễn đạt đầu ra giảng dạy như các giả thuyết cần kiểm tra. Khẳng định về niềm tin cần nghĩ-to-lên, giải thích hoặc bài tập dự đoán với sinh viên (Lu and Krishnamurthi, 2024).

Cách xây dựng nhân.. Delta debugging làm việc trên hunks dòng, nên hai thay đổi trên cùng một dòng không tách được và một bản sửa tối thiểu có thể kèm một sửa đổi trình bày cạnh một sửa lỗi logic. Bộ phân loại dựa trên luật và được sửa một lần sau khi audit các item huấn luyện và kiểm định (sửa đổi A6); cả hai lần audit do một trợ lý AI thực hiện, không phải gán nhãn độc lập của con người. Việc loại các bản sửa đa loại (MULTI, 54,4% sự kiện sửa) giữ cho nhân chuẩn chính xác nhưng làm nó nghiêng về các cơ chế đơn giản, thường ở mức output. Chúng tôi coi các lần nộp `sub_NNN` là theo thứ tự thời gian trong cùng sinh viên, bài tập và năm.

Mutant tiêm lỗi.. Mutant sạch hơn lỗi thật, phân bố loại do chúng tôi chọn, và các đặc trưng dựa trên mã xem xét đúng phần cú pháp bị sửa (sửa đổi A4). Đánh giá bằng đột biến cho biết một biểu diễn *có thể* tách được gì (Just et al., 2014), không cho biết các cơ chế xuất hiện thường xuyên thể nào trong lớp.

Phạm vi.. Một khóa học, một chuẩn ngôn ngữ (C90), một chính sách chấm (khớp output chính xác) và một cơ sở đào tạo. Các mô tả độ lệch được thiết kế không phụ thuộc bài, nhưng giá trị của chúng phụ thuộc vào việc chấm theo output; các khóa học dùng test theo thuộc tính hoặc unit test có thể thấy mẫu khác. Partition kiểm tra niêm phong của benchmark thật quá nhỏ cho thống kê phân cụm cấp cohort, đó là lý do sửa đổi A3 đánh giá cohort đầy đủ; quy nạp luật luôn dùng sinh viên được giữ lại.

Thống kê.. Các cohort khác nhau về kích thước và cân bằng nhân; ARI nhạy với loại chiếm đa số. Chúng tôi báo cáo khoảng bootstrap cấp cohort, kiểm định Wilcoxon có hiệu chỉnh Holm và k chọn bằng silhouette như phân tích phụ, nhưng số cohort (vài chục) hạn chế độ mạnh thống kê, và các item trong một cohort không độc lập giữa các biểu diễn của cùng một chương trình.

7. Kết luận

Gom cụm các chương trình sai theo những test mà chúng trượt là một cách hấp dẫn để cung cấp phản hồi vĩ mô cho giảng viên, và luật NẾU–THÌ của nó đọc lên như lời khuyên giảng dạy. Khi được đánh giá bằng các bản sửa tối thiểu đã kiểm chứng của chính sinh viên và bằng các mutant một cơ chế có kiểm soát, OAV theo test case hóa ra trộn lẫn cơ chế một cách có hệ thống: nó nhận ra các lỗi ở mức output chiếm đa số và gộp lẫn phần lớn phần còn lại. Bằng chứng không cần nhãn về cách output lệch khỏi đáp án khôi phục được nhiều hơn, rõ nhất khi các cơ chế cân bằng, và hỗ trợ các luật chuyển được giữa các bài tập; ngược lại, embedding mã đa dụng gom chương trình theo nguồn gốc. Vì mọi nhãn trong các benchmark của chúng tôi đều suy ra từ bằng chứng đã thực thi và có thể suy lại, các benchmark có thể được dùng lại để đánh giá biểu diễn mới, bộ chẩn đoán dựa trên LLM hoặc các test chủ động.

Ba bước sẽ củng cố các kết luận này: một audit độc lập bởi hai người chấm, tái lập trên các khóa học, ngôn ngữ và chính sách chấm khác, và các test thích ứng phá vỡ những va chạm chữ ký còn lại trước khi một giả thuyết đến tay giảng viên. Khẳng định về niềm tin của sinh viên cần nghiên cứu với sinh viên; các công cụ công bố ở đây nhằm làm cho những nghiên cứu đó dễ hơn khi thiết kế.

Phụ lục A. Protocol đã đăng ký và các sửa đổi

Protocol (`research/protocol-mechanism-v1.json`) được commit lúc 2026-09-30 12:06:00 UTC, trước mọi đánh giá cơ chế. Các sửa đổi A1–A6 được ghi trước khi xem bất kỳ kết quả đánh giá nào và A7 được ghi trước lần chạy kiểm tra niêm phong; mỗi sửa đổi đều có ghi ngày trong tệp protocol.

- A1** H1 ban đầu so trần của biểu diễn độ lệch với biểu diễn kết quả test. Vì biểu diễn độ lệch mịn hơn biểu diễn kết quả test, phép so sánh đó không thể thất bại; nó được thay bằng H1a (trần tuyệt đối theo kết quả test) và H1b (mức tăng so với phân hoạch ngẫu nhiên cùng độ mịn).
- A2** Mỗi sự kiện sửa một item: lần nộp sai cuối cùng trước lần nộp sửa, vì các lần nộp sai trước đó của cùng sinh viên gần như trùng lặp.
- A3** Nếu benchmark thật có ít hơn mười cohort kiểm tra đủ điều kiện, các giả thuyết phân cụm của nó dùng cohort đầy đủ theo bài tập, vì phân cụm không khớp gì trên nhãn. Điều kiện đã xảy ra (0 cohort kiểm tra đủ điều kiện).

- A4** Đặc trưng dựa trên mã xem xét đúng phần cú pháp mà toán tử tiềm lỗi sửa, nên điểm số của chúng trên bài tiềm lỗi là một cận lạc quan.
- A5** Loại các chương trình có kết quả khác nhau giữa hai lần chạy lại độc lập.
- A6** Bộ phân loại v1.1 sau khi audit nhãn các item huấn luyện và kiểm định (xem dưới). Một lượt kiểm định bắt đầu trước sửa đổi này đã bị dừng trước khi ghi bất kỳ kết quả nào.
- A7** Hệ số phạt ILA-2 được cố định ở 1,0, giá trị có macro-F1 kiểm định tốt nhất trên cả hai benchmark, và cách gán cụm theo từng item của cấu hình chính được lưu cho các phân tích khám phá theo loại.

Phụ lục B. Audit nhãn

Hai mẫu phân tầng gồm các item huấn luyện và kiểm định được trợ lý phân tích (một mô hình AI) xem xét; đây không phải gán nhãn độc lập của con người. Mẫu thứ nhất (64 ca, tối đa năm ca mỗi nhãn được gán) xác nhận cả 53 nhãn đơn loại đều đúng, và 6 trong 11 ca bị loại thực ra có một loại rõ ràng theo taxonomy (thêm giá trị khởi tạo vào khai báo, đổi kiểu bằng cách chuyển biến sang khai báo khác, hằng macro, và thiếu toàn bộ phần cài đặt). Bộ phân loại v1.1 xử lý các mẫu này; gán nhãn lại các bản sửa đã lưu làm thay đổi 25 nhãn chính. Mẫu thứ hai, không trùng mẫu đầu, gồm 29 ca của nhãn v1.1 và khớp 29 ca. Trên cả hai mẫu, nhãn đơn loại khớp 79/79 ca (cận dưới Wilson 95% là 0,954). Audit độc lập của hai người chấm là việc cần làm tiếp; các gói audit và quyết định được công bố.

Phụ lục C. Tái lập

Mọi bước chạy từ repository với Docker và Python 3.12:

```
docker build -t aai-c-replay:1 sandbox-replay
python scripts/replay_cpack.py --data DATA --split-lock SPLIT \
    --output REPLAY
python scripts/build_mechanism_benchmark.py --data DATA \
    --replay REPLAY --split-lock SPLIT --output BENCH_V1
python scripts/relabel_repairs.py --data DATA --bench BENCH_V1 \
    --audit AUDIT --output BENCH
python scripts/run_mechanism_eval.py --partition validation ...
python scripts/run_mechanism_eval.py --partition test \
    --scopes partition full --penalty 1,0 ...
```

Mỗi lượt chạy ghi dấu vân tay mã nguồn lúc bắt đầu, image container, hash đầu vào và hash chia dữ liệu. Lần chạy lại mất 14 phút trên máy ảo Docker bốn nhân; dựng benchmark đã thực thi 7.040 lần thử delta debugging và 4.350 mutant ứng viên.

Phụ lục D. Khôi phục theo từng loại

Phân tích khám phá (sửa đổi A7): tỷ lệ item mà cụm K-means (k theo nhãn, seed 42) chứa nó có loại của item là nhãn đa số.

Bảng D.1: Khôi phục theo loại trên bản sửa thật (cohort đầy đủ).

Loại	n	Chữ ký	Kết quả	Kết hợp+stdout	Độ lệch	Bảng chứng	Embedding
văn bản in ra	177	0,98	0,98		0,92	0,95	0,94
biểu thức tính	28	0,54	0,54		0,64	0,57	0,61
thiếu bước xử lý	27	0,37	0,37		0,67	0,56	0,63
điều kiện rẽ nhánh	17	0,29	0,24		0,65	0,59	0,59
biên vòng lặp	16	0,88	0,88		0,94	0,81	0,81
đặc tả định dạng in	13	0,00	0,00		0,69	0,69	0,69
luồng điều khiển	12	0,83	0,83		0,83	0,83	0,92
thừa bước xử lý	8	0,12	0,12		0,38	0,50	0,50
khởi tạo	7	0,57	0,57		1,00	1,00	1,00
đọc dữ liệu	6	0,50	0,50		0,50	0,50	0,50
vị trí câu lệnh	6	0,50	0,50		0,33	0,50	0,33
kiểu số	3	1,00	1,00		1,00	1,00	1,00

Bảng D.2: Khôi phục theo loại trên mutant tiềm lỗi (cohort kiểm tra).

Loại	n	Chữ ký	Kết quả	Kết hợp+stdout	Độ lệch	Bảng chứng	Embedding
thiếu bước xử lý	72	0,56	0,49		0,69	0,60	0,64
thừa bước xử lý	65	0,49	0,46		0,68	0,82	0,82
văn bản in ra	60	0,62	0,62		0,90	0,97	0,95
khởi tạo	53	0,40	0,40		0,60	0,53	0,58
biên vòng lặp	50	0,62	0,56		0,68	0,62	0,62
vị trí câu lệnh	48	0,12	0,08		0,67	0,79	0,58
biểu thức tính	43	0,51	0,51		0,70	0,67	0,77
điều kiện rẽ nhánh	29	0,38	0,34		0,45	0,45	0,48
đặc tả định dạng in	9	1,00	1,00		1,00	1,00	0,89

Tài liệu tham khảo

Al-Hossami, E., Bunescu, R., 2026. McMining: Automated discovery of misconceptions in student code, in: Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers), pp. 160–178.

- Altadmri, A., Brown, N.C.C., 2015. 37 million compilations: Investigating novice programming mistakes in large-scale student data, in: Proceedings of the 46th ACM Technical Symposium on Computer Science Education (SIGCSE), pp. 522–527. doi:10.1145/2676723.2677258.
- Aruleba, K., Oyelere, S.S., Agbo, F.J., Jere, N., 2026. From correctness to expert-refined misconception signals with human-in-the-loop validation in CS1 programming education, in: Proceedings of the 2026 United Kingdom and Ireland Computing Education Research Conference (UKICER). doi:10.1145/3830800.3830801.
- Ettles, A., Luxton-Reilly, A., Denny, P., 2018. Common logic errors made by novice programmers, in: Proceedings of the 20th Australasian Computing Education Conference (ACE '18), pp. 83–89. doi:10.1145/3160489.3160493.
- Glassman, E.L., Scott, J., Singh, R., Guo, P.J., Miller, R.C., 2015. Over-Code: Visualizing variation in student solutions to programming problems at scale. *ACM Transactions on Computer-Human Interaction* 22, 7:1–7:35. doi:10.1145/2699751.
- Gulwani, S., Radiček, I., Zuleger, F., 2018. Automated clustering and program repair for introductory programming assignments, in: Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), pp. 465–480. doi:10.1145/3192366.3192387.
- Gusukuma, L., Bart, A.C., Kafura, D., Ernst, J., 2018. Misconception-driven feedback: Results from an experimental study, in: Proceedings of the 2018 ACM Conference on International Computing Education Research (ICER), pp. 160–168. doi:10.1145/3230977.3231002.
- Head, A., Glassman, E., Soares, G., Suzuki, R., Figueredo, L., D’Antoni, L., Hartmann, B., 2017. Writing reusable code feedback at scale with mixed-initiative program synthesis, in: Proceedings of the Fourth ACM Conference on Learning @ Scale, pp. 89–98. doi:10.1145/3051457.3051467.
- Hermans, F., 2021. *The Programmer’s Brain*. Manning.
- Hoq, M., Pitts, G., Bhatt, T., Pandya, A., Lan, A., Brusilovsky, P., Akram, B., 2026. Pattern-based knowledge component extraction from student code using representation learning, in: Proceedings of the 19th International Conference on Educational Data Mining (EDM). doi:10.5281/zenodo.21040127.

- Hoq, M., Rao, A., Jaishankar, R., Piryani, K., Janapati, N., Vandenberg, J., Mott, B., Norouzi, N., Lester, J., Akram, B., 2025. Automated identification of logical errors in programs: Advancing scalable analysis of student misconceptions, in: Proceedings of the 18th International Conference on Educational Data Mining (EDM).
- Hubert, L., Arabie, P., 1985. Comparing partitions. *Journal of Classification* 2, 193–218. doi:10.1007/BF01908075.
- Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G., 2014. Are mutants a valid substitute for real faults in software testing?, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), pp. 654–665. doi:10.1145/2635868.2635929.
- Kaleeswaran, S., Santhiar, A., Kanade, A., Gulwani, S., 2016. Semi-supervised verified feedback generation, in: Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), pp. 739–750. doi:10.1145/2950290.2950363.
- Li, C., Tang, Z., Wang, J., Wu, Y., Hashimoto, K., Gao, L., 2026. PyMETA: Evaluating student code diagnosis on and beyond the first execution error. *arXiv:2606.30610*. preprint, revised 15 September 2026.
- Liding, Z., Osmolovskiy, M., Lanka, H., Roy, N., Borela, R., 2026. AI-augmented instruction: Real-time misconception detection, in: Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.2. doi:10.1145/3770761.3777326.
- Lu, K.C., Krishnamurthi, S., 2024. Identifying and correcting programming language behavior misconceptions, in: Proceedings of the ACM on Programming Languages (OOPSLA).
- Niousha, R., Boatright Smith, S., O’Neill, A., Zamfirescu-Pereira, J.D., DeNero, J., Norouzi, N., 2026. Misconception-aware LLM programming tutor: Lessons learned from student-tutor interactions, in: Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.2. doi:10.1145/3770761.3777289.
- Orvalho, P., Janota, M., Manquinho, V., 2024. C-Pack of IPAs: A C90 program benchmark of introductory programming assignments, in: Proceedings of the 5th International Workshop on Automated Program Repair (APR@ICSE 2024), pp. 14–21. doi:10.1145/3643788.3648010.

- Orvalho, P., Janota, M., Manquinho, V., 2025. InvAASTCluster: On applying invariant-based program clustering to introductory programming assignments. *Journal of Systems and Software* .
- Orvalho, P., Janota, M., Manquinho, V., 2026. MENTOR: Fixing introductory programming assignments with formula-based fault localization and LLM-driven program repair. *Journal of Systems and Software* .
- Piech, C., Huang, J., Nguyen, A., Phulsuksombati, M., Sahami, M., Guibas, L., 2015. Learning program embeddings to propagate feedback on student code, in: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pp. 1093–1102.
- Prasad, S., Greenman, B., Nelson, T., Krishnamurthi, S., 2024. Conceptual mutation testing for student programming misconceptions. *The Art, Science, and Engineering of Programming* 8, 7.
- Qian, Y., Lehman, J., 2017. Students’ misconceptions and other difficulties in introductory programming: A literature review. *ACM Transactions on Computing Education* 18, 1:1–1:24. doi:10.1145/3077618.
- Shi, Y., Shah, K., Wang, W., Marwan, S., Penmetsa, P., Price, T.W., 2021. Toward semi-automatic misconception discovery using code embeddings, in: *Proceedings of the 11th International Learning Analytics and Knowledge Conference (LAK)*, pp. 606–612. doi:10.1145/3448139.3448205.
- Sorva, J., 2013. Notional machines and introductory programming education. *ACM Transactions on Computing Education* 13, 8:1–8:31. doi:10.1145/2483710.2483713.
- Suzuki, R., Soares, G., Head, A., Glassman, E., Reis, R., Mongiovi, M., D’Antoni, L., Hartmann, B., 2017. TraceDiff: Debugging unexpected code behavior using trace divergences, in: *IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*.
- Tolun, M.R., Abu-Soud, S.M., 1998. ILA: An inductive learning algorithm for rule extraction. *Expert Systems with Applications* 14, 361–370. doi:10.1016/S0957-4174(97)00089-4.
- Tolun, M.R., Sever, H., Uludag, M., Abu-Soud, S.M., 1999. ILA-2: An inductive learning algorithm for knowledge discovery. *Cybernetics and Systems* 30, 609–628. doi:10.1080/019697299125037.

- Vinh, N.X., Epps, J., Bailey, J., 2010. Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. *Journal of Machine Learning Research* 11, 2837–2854.
- Yi, J., Ahmed, U.Z., Karkare, A., Tan, S.H., Roychoudhury, A., 2017. A feasibility study of using automated program repair for introductory programming assignments. doi:10.1145/3106237.3106262. eSEC/FSE 2017.
- Zeller, A., Hildebrandt, R., 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering* 28, 183–200. doi:10.1109/32.988498.
- Zhang, Y., et al., 2025. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. **arXiv:2506.05176**.